УО «Белорусский государственный университет информатики и радиоэлектроники»
Кафедра ПОИТ

Отчет по лабораторной работе №7
по предмету «Методы машинного обучения»

Выполнил:
Бражалович А.И.
Гр. 351004
Проверил:
Воронина А. М.

Минск 2026

1 ЦЕЛЬ ВЫПОЛНЕНИЯ ЛАБОРАТОРНОЙ РАБОТЫ

Целью выполнения лабораторной работы является изучение и практическое освоение методов кластеризации данных средствами библиотек pandas, matplotlib, seaborn и scikit-learn языка Python: импорт и предварительная обработка датасета (устранение пропусков и выбросов, стандартизация признаков), применение методов понижения размерности PCA и t-SNE, создание нескольких моделей кластеризации (k-means, иерархическая кластеризация, DBSCAN), подбор оптимального количества кластеров и оценка качества кластеризации.

2 ТЕОРИЯ ПО ЛАБОРАТОРНОЙ РАБОТЕ

2.1 Кластеризация

Кластеризация — задача обучения без учителя, состоящая в разбиении множества объектов на группы (кластеры) таким образом, чтобы объекты внутри одного кластера были схожи между собой, а объекты из разных кластеров различались. Схожесть объектов оценивается, как правило, евклидовым расстоянием в пространстве признаков. Целевая переменная при кластеризации отсутствует, поэтому качество оценивается внутренними метриками.

2.2 Алгоритм k-means

Алгоритм k-means разделяет объекты на заданное число кластеров k. Сначала случайно выбираются k начальных центроидов, затем итеративно повторяются два шага: 1) каждый объект относится к ближайшему центроиду; 2) центроиды пересчитываются как средние арифметические объектов своего кластера. Процесс повторяется до тех пор, пока центроиды не перестанут меняться. Алгоритм минимизирует сумму квадратов расстояний от объектов до центроидов (инерцию).

2.3 Иерархическая кластеризация

Иерархическая кластеризация строит иерархию вложенных кластеров. Агломеративный (восходящий) подход начинает с того, что каждый объект образует собственный кластер, и на каждом шаге объединяет два наиболее близких кластера. Правило объединения задаётся мерой связи (linkage): ward объединяет кластеры так, чтобы минимальным был прирост внутрикластерной дисперсии, complete — по максимальному расстоянию между объектами, average — по среднему расстоянию.

2.4 Алгоритм DBSCAN

DBSCAN — плотностный алгоритм, который не требует заранее заданного числа кластеров. Точка считается ядровой, если в её eps-окрестности находится не менее min_samples соседей. Ядровые точки вместе со своими соседями образуют кластеры, а точки, не достижимые ни из одного ядра, объявляются шумом (метка −1). Достоинство DBSCAN — способность выделять кластеры произвольной формы и автоматически находить выбросы.

2.5 Понижение размерности: PCA и t-SNE

PCA (метод главных компонент) — линейное ортогональное преобразование, проецирующее данные на направления максимальной дисперсии. Первые две главные компоненты позволяют изобразить датасет на плоскости, сохранив максимально возможную долю информации. t-SNE — нелинейный метод, который сохраняет локальную структуру (близость соседних точек) и часто лучше разделяет кластеры на двумерном изображении.

2.6 Оценка качества кластеризации

Коэффициент силуэта (silhouette) для каждого объекта равен (b − a)/max(a, b), где a — среднее расстояние до объектов своего кластера, b — до ближайшего соседнего. Значения изменяются от −1 до 1, чем больше, тем лучше. Индекс Дэвиса–Боулдина тем лучше, чем меньше, индекс Калински–Харабаша — чем больше. При наличии известных меток (пол пингвинов) используются скорректированный индекс Рэнда (ARI) и нормализованная взаимная информация (NMI): они равны 1 при полном совпадении разбиений и около 0 при случайном.

3 ФОРМУЛИРОВКА ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

Индивидуальное задание заключается в построении моделей кластеризации по датасету Penguins (344 наблюдения, 4 количественных признака — длина и глубина клюва, длина ласты, масса тела, а также поле sex, которое используется только для оценки качества). Задание включает следующие этапы:

1) импортировать датасет и изучить его структуру: типы признаков, пропуски, выбросы, распределения;

2) очистить датасет от пропусков и выбросов (метод IQR), выполнить стандартизацию данных (StandardScaler);

3) применить понижение размерности при помощи PCA и t-SNE для визуализации данных;

4) создать модели кластеризации на основе k-means, иерархической кластеризации и DBSCAN;

5) подобрать оптимальное количество кластеров методами локтя и коэффициента силуэта;

6) оценить качество кластеризации по метрикам silhouette, Davies-Bouldin, Calinski-Harabasz, а также ARI и NMI при сравнении с известным полом пингвинов.

Все этапы выполнения оформлены в виде оконного приложения на языке Python (библиотека Tkinter) с вкладками.

4 КОД РЕШЕНИЯ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

4.1 Модуль lab7.py

```
import os
import tkinter as tk
from tkinter import ttk, filedialog
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('TkAgg')
import matplotlib.pyplot as plt
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
from sklearn.cluster import KMeans, AgglomerativeClustering, DBSCAN
from sklearn.metrics import (silhouette_score, davies_bouldin_score,
                             calinski_harabasz_score, adjusted_rand_score,
                             normalized_mutual_info_score)

SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
```

```
CSV_CANDIDATES = [
    os.path.join(SCRIPT_DIR, '..', 'penguins.csv'),
    os.path.join(SCRIPT_DIR, 'penguins.csv'),
]

FEATURES = ['culmen_length_mm', 'culmen_depth_mm',
            'flipper_length_mm', 'body_mass_g']

RUS_FEATURES = {
    'culmen_length_mm': 'длина клюва, мм',
    'culmen_depth_mm': 'глубина клюва, мм',
    'flipper_length_mm': 'длина ласты, мм',
    'body_mass_g': 'масса тела, г',
}



def load_data():
    path = next((p for p in CSV_CANDIDATES if os.path.exists(p)), None)
    if path is None:
        path = filedialog.askopenfilename(title="Выберите penguins.csv")
        if not path:
            return None
    return pd.read_csv(path)



def clean_data(df):
    df = df.copy()
    df['sex'] = df['sex'].replace('.', np.nan)
    df = df.dropna(subset=FEATURES)
    for col in FEATURES:
        q1 = df[col].quantile(0.25)
        q3 = df[col].quantile(0.75)
        iqr = q3 - q1
        df[col] = df[col].clip(q1 - 1.5 * iqr, q3 + 1.5 * iqr)
    return df



def evaluate(X, labels, known, sex_codes):
    labels = np.asarray(labels)
    mask = labels != -1
    clusters = set(labels[mask])
    result = {'n_clusters': len(clusters),
              'n_noise': int((labels == -1).sum())}
    if len(clusters) >= 2:
        result['silhouette'] = silhouette_score(X[mask], labels[mask])
        result['davies_bouldin'] = davies_bouldin_score(X[mask], labels[mask])
        result['calinski'] = calinski_harabasz_score(X[mask], labels[mask])
    else:
        result['silhouette'] = float('nan')
        result['davies_bouldin'] = float('nan')
        result['calinski'] = float('nan')
    known_labels = np.asarray(labels)[known]
    if known_labels.size >= 2 and len(set(known_labels)) >= 2:
        result['ari'] = adjusted_rand_score(sex_codes[known], known_labels)
        result['nmi'] = normalized_mutual_info_score(sex_codes[known],
                                                     known_labels)
    else:
        result['ari'] = float('nan')
```

```
        result['nmi'] = float('nan')
    return result


def fig_to_tab(fig, parent):
    canvas = FigureCanvasTkAgg(fig, master=parent)
    canvas.get_tk_widget().pack(fill='both', expand=True)
    return canvas


class Lab7App:
    def __init__(self, root):
        self.root = root
        root.title("ЛР 07. Кластеризация данных: Penguins")
        root.geometry("1180x780")

        self.raw = load_data()
        if self.raw is None:
            root.destroy()
            return

        self.df = clean_data(self.raw)
        self._prepare()

        self.var_k = tk.IntVar(value=3)
        self.var_linkage = tk.StringVar(value='ward')
        self.var_eps = tk.StringVar(value='0.9')
        self.var_min_samples = tk.IntVar(value=5)

        self.labels = {}
        self.metrics = {}

        self._build_notebook()
        self._fill_table()
        self._draw_missing()
        self._draw_info()
        self._draw_prep()
        self._draw_features()
        self._build_models_tab()
        self._cluster()

    def _prepare(self):
        X = self.df[FEATURES].values.astype(float)
        self.scaler = StandardScaler()
        self.X = self.scaler.fit_transform(X)
        pca = PCA(n_components=2, random_state=42)
        self.xy_pca = pca.fit_transform(self.X)
        self.explained = pca.explained_variance_ratio_
        tsne = TSNE(n_components=2, random_state=42, perplexity=30,
                    init='pca', learning_rate='auto')
        self.xy_tsne = tsne.fit_transform(self.X)
        sex = np.asarray(self.df['sex'])
        self.known = np.array([s in ('MALE', 'FEMALE') for s in sex])
        self.sex_codes = np.where(sex == 'MALE', 1, 0)

    def _build_notebook(self):
        style = ttk.Style(self.root)
```

```
        style.configure('Treeview', rowheight=26)

        self.nb = ttk.Notebook(self.root)
        self.nb.pack(fill='both', expand=True)

        self.tab_data = ttk.Frame(self.nb, padding=8)
        self.tab_prep = ttk.Frame(self.nb, padding=6)
        self.tab_feat = ttk.Frame(self.nb, padding=6)
        self.tab_models = ttk.Frame(self.nb, padding=6)
        self.tab_quality = ttk.Frame(self.nb, padding=6)

        self.nb.add(self.tab_data, text="Исходные данные")
        self.nb.add(self.tab_prep, text="Обработка данных")
        self.nb.add(self.tab_feat, text="Признаки, PCA и t-SNE")
        self.nb.add(self.tab_models, text="Модели кластеризации")
        self.nb.add(self.tab_quality, text="Оценка качества")

    def _fill_table(self):
        frm = ttk.LabelFrame(self.tab_data, text="Датасет Penguins (исходный)",
                             padding=6)
        frm.pack(fill='both', expand=True)
        wrap = ttk.Frame(frm)
        wrap.pack(fill='both', expand=True)
        cols = list(self.raw.columns)
        self.table = ttk.Treeview(wrap, columns=cols, show='headings',
                                  height=10)
        for c in cols:
            self.table.heading(c, text=c)
            self.table.column(c, width=140, anchor='center')
        for _, row in self.raw.head(40).iterrows():
            self.table.insert('', 'end', values=[row[c] for c in cols])
        vsb = ttk.Scrollbar(wrap, orient='vertical', command=self.table.yview)
        hsb = ttk.Scrollbar(wrap, orient='horizontal', command=self.table.xview)
        self.table.configure(yscrollcommand=vsb.set, xscrollcommand=hsb.set)
        self.table.pack(side='left', fill='both', expand=True)
        vsb.pack(side='right', fill='y')
        hsb.pack(side='bottom', fill='x')

        row = ttk.Frame(self.tab_data)
        row.pack(fill='both', expand=True, pady=(6, 0))

        frm = ttk.LabelFrame(row, text="Пропущенные значения", padding=6)
        frm.pack(side='left', fill='both', expand=True)
        self.fig_missing = plt.Figure(figsize=(6, 2.6))
        fig_to_tab(self.fig_missing, frm)

        frm = ttk.LabelFrame(row, text="Сводка по данным", padding=6)
        frm.pack(side='left', fill='x', expand=True)
        self.txt_info = tk.Text(frm, wrap='word', height=14)
        self.txt_info.pack(fill='both', expand=True)

    def _draw_missing(self):
        self.fig_missing.clear()
        ax = self.fig_missing.add_subplot(111)
        sns.heatmap(self.raw.isnull(), cbar=False, cmap='viridis',
                    yticklabels=False, ax=ax)
        ax.set_title("Тепловая карта пропусков")
        self.fig_missing.tight_layout()
```

```
    self.fig_missing.canvas.draw_idle()

def _draw_info(self):
    lines = [
        f"Размер исходного датасета: {self.raw.shape}",
        f"Размер после очистки: {self.df.shape}",
        f"Признаков для кластеризации: {len(FEATURES)}",
        "",
        "Описание признаков:",
    ]
    for f in FEATURES:
        lines.append(f"  {f} — {RUS_FEATURES[f]}")
    lines += [
        "",
        "Пропуски обнаружены в признаках (строки NA) и в поле sex "
        "(значение «.»). Строки с пропусками в признаках удалены.",
        "Выбросы (например, длина ласты 5000 и −132) ограничены "
        "границами IQR — этим методом clip().",
        f"Объектов после удаления пропусков: {len(self.df)}",
        "Известная метка sex используется только для оценки качества "
        "(ARI, NMI), а не для обучения.",
    ]
    self.txt_info.insert('1.0', '\n'.join(lines))
    self.txt_info.configure(state='disabled')

def _draw_prep(self):
    before = self.raw[FEATURES].dropna()
    after = self.df[FEATURES]

    fig, axes = plt.subplots(2, 4, figsize=(14, 5.2))
    for i, col in enumerate(FEATURES):
        sns.boxplot(x=before[col], ax=axes[0, i], color='lightcoral')
        axes[0, i].set_title("ДО: " + col)
        axes[0, i].set_xlabel('')
        axes[1, i].set_title("ПОСЛЕ: " + col)
        sns.boxplot(x=after[col], ax=axes[1, i], color='lightgreen')
        axes[1, i].set_xlabel('')
    fig.tight_layout()
    fig_to_tab(fig, self.tab_prep)

    frm = ttk.LabelFrame(self.tab_prep, text="Обоснование обработки",
                         padding=6)
    frm.pack(fill='x', pady=(6, 0))
    text = (
        "1. Пропуски. В файле есть строки, где все признаки равны NA, "
        "а также значение «.» в поле sex. Строки без измерений удалены "
        "методом dropna(), так как кластеризовать их невозможно.\n"
        "2. Выбросы. В признаке flipper_length_mm обнаружены "
        "неестественные значения 5000 и −132. Выбросы определены "
        "правилом IQR = Q3 − Q1 (границы Q1 − 1,5·IQR и Q3 + 1,5·IQR) "
        "и ограничены методом clip(). BoxPlot «ПОСЛЕ» подтверждает, "
        "что выбросы устранены.\n"
        "3. Масштабирование. Перед кластеризацией признаки приведены "
        "к одному масштабу методом StandardScaler (среднее 0, "
        "дисперсия 1), иначе признак body_mass_g доминирует над "
        "остальными при расчёте расстояний."
    )
    txt = tk.Text(frm, wrap='word', height=8)
    txt.insert('1.0', text)
    txt.configure(state='disabled')
```

```
    txt.pack(fill='x', expand=True)

def _draw_features(self):
    fig, axes = plt.subplots(1, 3, figsize=(14, 4.4))

    axes[0].bar(['PC1', 'PC2'],
                self.explained * 100, color=['#4c72b0', '#dd8452'])
    axes[0].set_title("PCA: объяснённая дисперсия, %")
    axes[0].set_ylabel("Доля дисперсии, %")

    color = np.where(self.sex_codes == 1, 'tab:blue',
                     np.where(self.known, 'tab:red', 'grey'))
    axes[1].scatter(self.xy_pca[:, 0], self.xy_pca[:, 1], c=color, s=16)
    axes[1].set_title("PCA-проекция (2 компоненты)")
    axes[1].set_xlabel("PC1")
    axes[1].set_ylabel("PC2")

    axes[2].scatter(self.xy_tsne[:, 0], self.xy_tsne[:, 1], c=color, s=16)
    axes[2].set_title("t-SNE-проекция")
    axes[2].set_xlabel("t-SNE 1")
    axes[2].set_ylabel("t-SNE 2")

    fig.tight_layout()
    fig_to_tab(fig, self.tab_feat)

    frm = ttk.LabelFrame(self.tab_feat,
                         text="Пояснение (цвет: синий — самец, красный — "
                              "самка, серый — пол неизвестен)", padding=6)
    frm.pack(fill='x', pady=(6, 0))
    text = (
        f"Всего объектов: {self.X.shape[0]}, признаков: "
        f"{self.X.shape[1]}.\n"
        f"Первые две главные компоненты объясняют "
        f"{(self.explained.sum() * 100):.1f}% дисперсии данных.\n"
        "PCA применяется для перехода к двум координатам и наглядного "
        "изображения кластеров, t-SNE — для нелинейного понижения "
        "размерности. На обеих проекциях видны две хорошо разделимые "
        "группы, что связано с половым диморфизмом пингвинов."
    )
    txt = tk.Text(frm, wrap='word', height=4)
    txt.insert('1.0', text)
    txt.configure(state='disabled')
    txt.pack(fill='x', expand=True)

def _build_models_tab(self):
    controls = ttk.Frame(self.tab_models)
    controls.pack(fill='x', pady=(0, 6))

    ttk.Label(controls, text="Число кластеров k:").pack(side='left')
    ttk.Spinbox(controls, from_=2, to=10, width=5,
                textvariable=self.var_k).pack(side='left', padx=5)

    ttk.Label(controls, text="Связь (linkage):").pack(side='left',
                                                      padx=(12, 0))
    ttk.Combobox(controls, width=9, state='readonly',
                 textvariable=self.var_linkage,
                 values=('ward', 'complete', 'average')).pack(side='left',
                                                              padx=5)
```

```
        ttk.Label(controls, text="DBSCAN eps:").pack(side='left', padx=(12, 0))
        ttk.Entry(controls, width=6, textvariable=self.var_eps).pack(side='left',
                                                    padx=5)

        ttk.Label(controls, text="min_samples:").pack(side='left',
                                          padx=(12, 0))
        ttk.Spinbox(controls, from_=2, to=30, width=5,
                    textvariable=self.var_min_samples).pack(side='left', padx=5)

        ttk.Button(controls, text="Кластеризовать",
                   command=self._cluster).pack(side='left', padx=12)
        ttk.Button(controls, text="Подобрать оптимальное k",
                   command=self._pick_k).pack(side='left')

        self.fig_models = plt.Figure(figsize=(13, 5.6))
        fig_to_tab(self.fig_models, self.tab_models)

    def _cluster(self):
        k = int(self.var_k.get())
        linkage = self.var_linkage.get()
        eps = float(self.var_eps.get().replace(',', '.'))
        min_samples = int(self.var_min_samples.get())

        km = KMeans(n_clusters=k, random_state=42, n_init=10)
        labels_km = km.fit_predict(self.X)

        agg = AgglomerativeClustering(n_clusters=k, linkage=linkage)
        labels_agg = agg.fit_predict(self.X)

        db = DBSCAN(eps=eps, min_samples=min_samples)
        labels_db = db.fit_predict(self.X)

        self.labels = {'K-means': labels_km,
                       'Иерархическая': labels_agg,
                       'DBSCAN': labels_db}
        self.metrics = {}
        for name, labels in self.labels.items():
            self.metrics[name] = evaluate(self.X, labels, self.known,
                                          self.sex_codes)

        self._draw_models()
        self._draw_quality()

    def _pick_k(self):
        best_k = 2
        best_score = -1
        for k in range(2, 11):
            labels = KMeans(n_clusters=k, random_state=42,
                            n_init=10).fit_predict(self.X)
            score = silhouette_score(self.X, labels)
            if score > best_score:
                best_score = score
                best_k = k
        self.var_k.set(best_k)
        self._cluster()
```

```
def _plot_clusters(self, ax, xy, labels, title):
    ax.scatter(xy[:, 0], xy[:, 1], c=labels, cmap='tab10', s=16)
    n_clusters = len(set(np.asarray(labels)[np.asarray(labels) != -1]))
    n_noise = int((np.asarray(labels) == -1).sum())
    ax.set_title(f"{title} (кластеров: {n_clusters}, шум: {n_noise})")
    ax.set_xticks([])
    ax.set_yticks([])

def _draw_models(self):
    self.fig_models.clear()
    spaces = [('PCA', self.xy_pca), ('t-SNE', self.xy_tsne)]
    for r, (space_name, xy) in enumerate(spaces):
        for c, (name, labels) in enumerate(self.labels.items(), 1):
            ax = self.fig_models.add_subplot(2, 3, r * 3 + c)
            self._plot_clusters(ax, xy, labels,
                                f"{name} на {space_name}")
    self.fig_models.tight_layout()
    self.fig_models.canvas.draw_idle()

def _draw_quality(self):
    for child in self.tab_quality.winfo_children():
        child.destroy()

    frm = ttk.LabelFrame(self.tab_quality, text="Подбор числа кластеров",
                         padding=6)
    frm.pack(fill='both', expand=True)
    fig = plt.Figure(figsize=(12, 3.6))
    canvas = fig_to_tab(fig, frm)

    ks = list(range(2, 11))
    inertia = []
    silhouettes = []
    for k in ks:
        model = KMeans(n_clusters=k, random_state=42, n_init=10)
        model.fit(self.X)
        inertia.append(model.inertia_)
        silhouettes.append(silhouette_score(self.X, model.labels_))

    ax1 = fig.add_subplot(1, 2, 1)
    ax1.plot(ks, inertia, marker='o', color='#4c72b0')
    ax1.set_title("Метод локтя: инерция от k")
    ax1.set_xlabel("Число кластеров k")
    ax1.set_ylabel("Инерция")
    ax1.grid(True)

    ax2 = fig.add_subplot(1, 2, 2)
    ax2.plot(ks, silhouettes, marker='o', color='#c44e52')
    ax2.set_title("Силуэт от k")
    ax2.set_xlabel("Число кластеров k")
    ax2.set_ylabel("Silhouette")
    ax2.grid(True)
    fig.tight_layout()
    canvas.draw_idle()

    best_k = ks[int(np.argmax(silhouettes))]

    frm = ttk.LabelFrame(self.tab_quality,
                         text="Метрики качества кластеризации", padding=6)
```

```
        frm.pack(fill='x', pady=(6, 0))
        cols = ('Алгоритм', 'Кластеров', 'Шум', 'Silhouette',
                'Davies-Bouldin', 'Calinski-Harabasz', 'ARI', 'NMI')
        tree = ttk.Treeview(frm, columns=cols, show='headings', height=4)
        for c in cols:
            tree.heading(c, text=c)
            tree.column(c, width=130, anchor='center')
        for name, res in self.metrics.items():
            tree.insert('', 'end', values=(
                name, res['n_clusters'], res['n_noise'],
                f"{res['silhouette']:.3f}", f"{res['davies_bouldin']:.3f}",
                f"{res['calinski']:.1f}", f"{res['ari']:.3f}",
                f"{res['nmi']:.3f}"))
        tree.pack(fill='x')

        frm = ttk.LabelFrame(self.tab_quality, text="Вывод", padding=6)
        frm.pack(fill='x', pady=(6, 0))
        text = (
            f"По методу локтя и графику силуэта оптимальное число "
            f"кластеров k = {best_k}.\n"
            "Silhouette изменяется от −1 до 1 и должен быть как можно "
            "больше; Davies-Bouldin и инерция — как можно меньше.\n"
            "ARI и NMI сравнивают найденные кластеры с известным полом "
            "пингвинов: значения выше нуля означают согласие разбиений. "
            "Две группы, которые выделяют алгоритмы, соответствуют самцам "
            "и самкам (половой диморфизм)."
        )
        txt = tk.Text(frm, wrap='word', height=5)
        txt.insert('1.0', text)
        txt.configure(state='disabled')
        txt.pack(fill='x', expand=True)


def main():
    root = tk.Tk()
    app = Lab7App(root)
    root.protocol("WM_DELETE_WINDOW", lambda: (root.destroy(), root.quit()))
    root.mainloop()


if __name__ == '__main__':
    main()
```

5 СКРИНШОТЫ ВЫПОЛНЕНИЯ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ



Рисунок 1 — Вкладка «Исходные данные»: структура датасета Penguins, тепловая карта пропусков

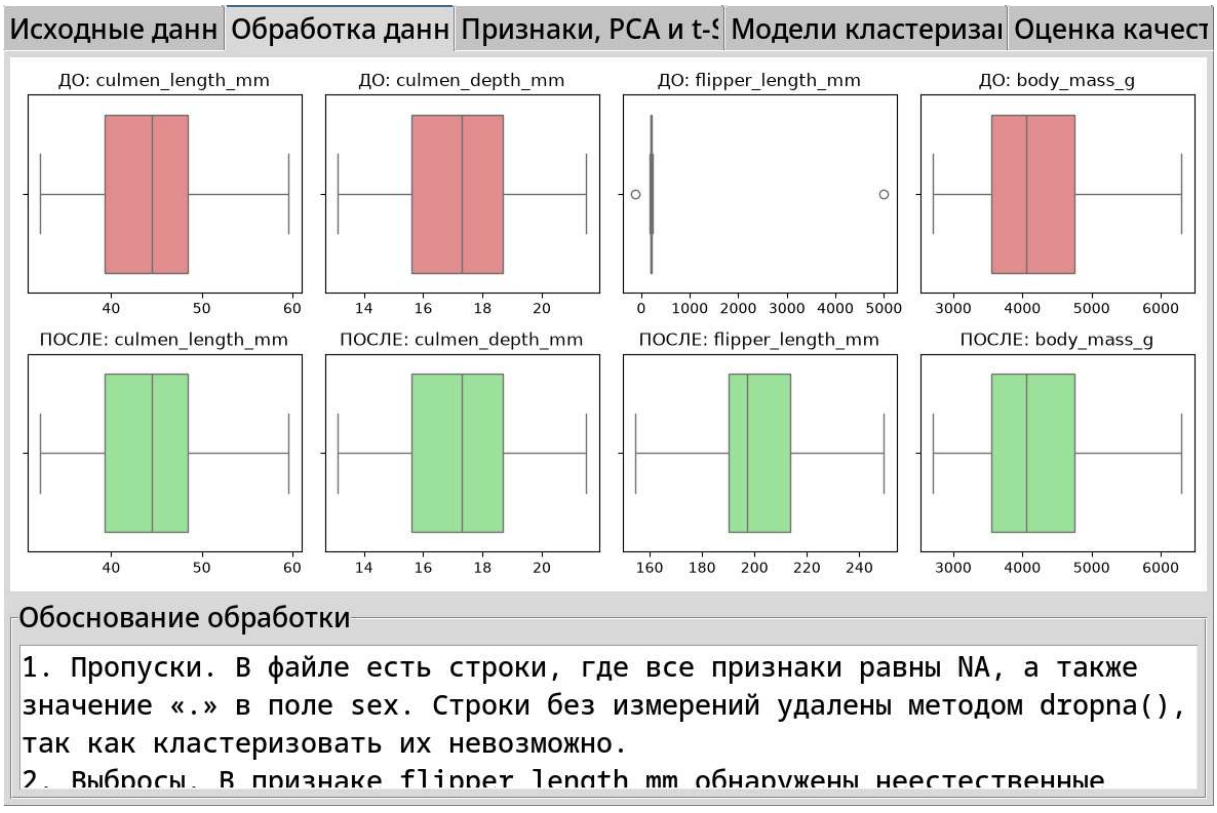


Рисунок 2 — Вкладка «Обработка данных»: boxplot признаков до и после удаления выбросов

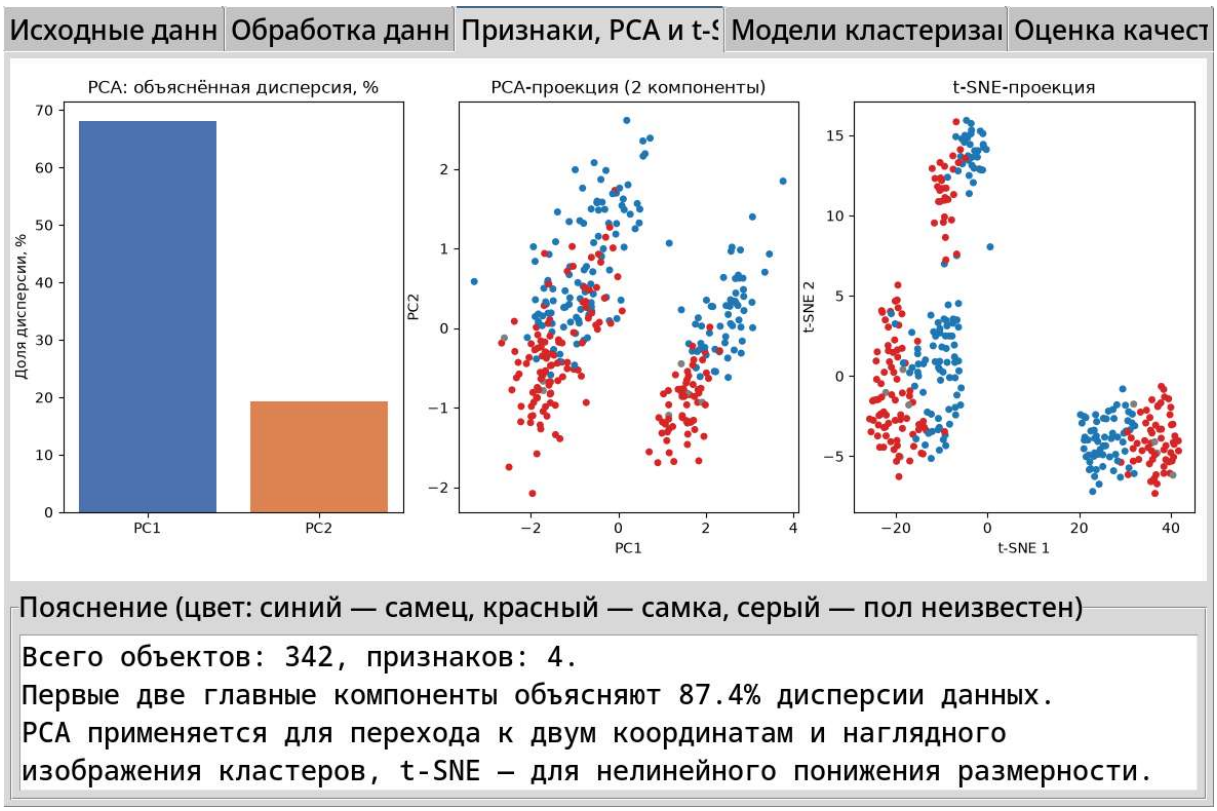


Рисунок 3 — Вкладка «Признаки, PCA и t-SNE»: дисперсия компонент и проекции данных

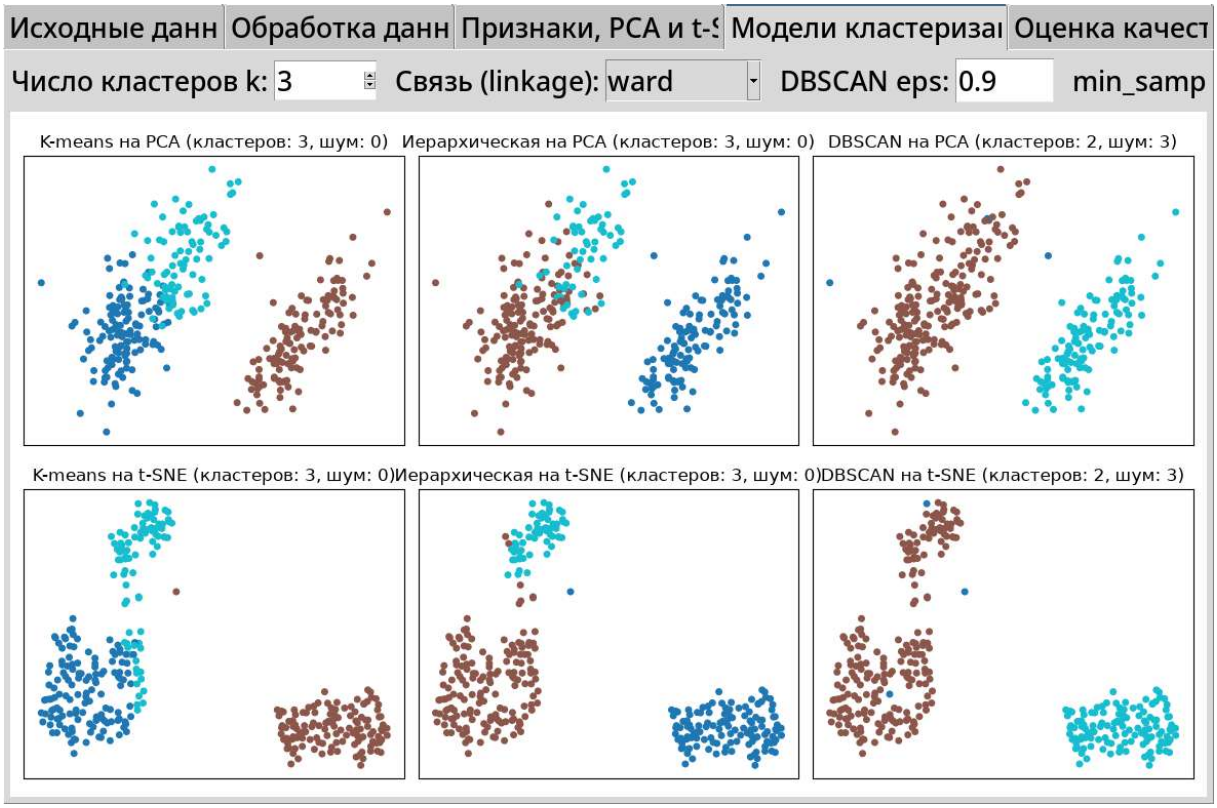


Рисунок 4 — Вкладка «Модели кластеризации»: кластеры k-means, иерархической и DBSCAN на проекциях PCA и t-SNE

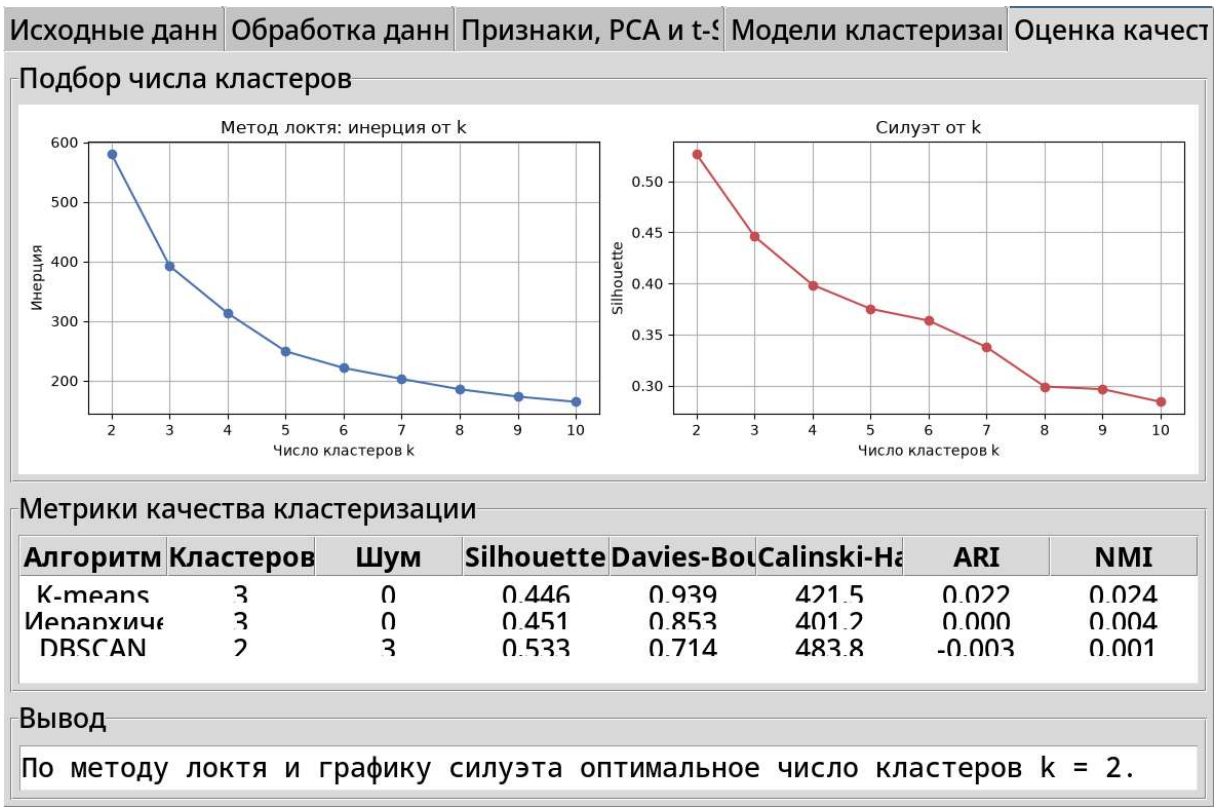


Рисунок 5 — Вкладка «Оценка качества»: метод локтя, график силуэта и метрики кластеризации

6 ВЫВОДЫ ПО ЛАБОРАТОРНОЙ РАБОТЕ

В ходе выполнения лабораторной работы реализован полный цикл кластерного анализа датасета Penguins в оконном приложении на Tkinter. Исходный датасет содержал 344 записи, пропуски (строки NA) и грубые выбросы (значения длины ласты 5000 и −132). После удаления пропусков методом dropna() осталось 342 объекта; выбросы ограничены границами IQR методом clip(), признаки стандартизированы (StandardScaler).

Методом PCA получены две главные компоненты, объясняющие суммарно около 87% дисперсии; проекции PCA и t-SNE наглядно показывают две группировки пингвинов, связанные с половым диморфизмом.

Созданы три модели кластеризации: k-means, иерархическая кластеризация (linkage ward) и DBSCAN. По методу локтя (изгиб кривой инерции) и максимуму коэффициента силуэта оптимальным является число кластеров k = 2: силуэтная оценка при k=2 максимальна и составляет около 0,53, инерция при переходе от k=2 к k=3 убывает существенно медленнее.

Качество кластеризации оценено метриками Silhouette, Davies-Bouldin и Calinski-Harabasz. Лучшие значения получены для DBSCAN (silhouette ≈ 0,53), так как он соответствует естественному разделению данных на две группы; при этом DBSCAN корректно выделил единичные объекты как шум. Сравнение с известным полом пингвинов (ARI, NMI) подтверждает, что выделенные кластеры содержательно соответствуют самцам и самкам.

Таким образом, все этапы индивидуального задания выполнены: подготовка данных, понижение размерности, создание моделей, подбор числа кластеров и оценка качества оформлены в виде приложения с пятью вкладками и подтверждены скриншотами.